

Python Algorithms

A Complete Curriculum — From Django Developer to Algorithm Expert

Prepared for Meseret

How to Use This Curriculum

This roadmap takes you from your existing Django/Python background to genuine algorithmic depth. It is organized into five phases, roughly a year of consistent, focused practice — because real expertise is built through repetition and real problem-solving, not shortcuts. Since you already know Python syntax, OOP, and how applications are structured, this curriculum skips basic syntax and goes straight into algorithmic thinking.

Recommended pace: 4 focused hours a day, 5-6 days a week. Each day should mix learning a concept with hands-on problem solving — the problem solving matters more than the reading.

The Five Phases

Phase	Duration	Focus
1. Foundation Sprint	2 weeks	Core data structures, Big-O, recursion, sorting/searching
2. Intermediate Patterns	4-6 weeks	Trees, graphs, DP, greedy, backtracking
3. Advanced Structures	6-8 weeks	Heaps, tries, union-find, advanced DP, interview prep
4. Applied Practice	8-12 weeks	Real projects, open source, contests, code reviews
5. Expert Track	Ongoing (6+ months)	Competitive programming, teaching, specialization

Phase 1 — Foundation Sprint (Days 1-14)

Goal: build real fluency in core data structures and algorithmic thinking. By day 14 you should comfortably solve Easy-to-Medium problems without help.

Day 1 — Big-O & Time Complexity

Learn

- What O(1), O(n), O(n^2), O(log n) mean and how to spot them in code

Practice

- Rewrite 3 simple functions and label their time complexity

Resource

- VisuAlgo (complexity) + GeeksforGeeks article

Day 2 — Arrays & Strings

Learn

- Traversal, slicing, in-place modification, common patterns

Practice

- 5 LeetCode Easy problems (tag: Array, String) — e.g. reverse a string, find max/min, remove duplicates

Day 3 — Hashmaps & Sets

Learn

- When to use a dict/set instead of a list for O(1) lookups

Practice

- 5 LeetCode Easy problems (tag: Hash Table) — e.g. Two Sum, first non-repeating character

Day 4 — Two-Pointer & Sliding Window

Learn

- Solving array/string problems without nested loops

Practice

- 4-5 problems — e.g. reverse array in-place, max subarray sum of size k

Day 5 — Stacks & Queues

Learn

- LIFO/FIFO, when each is useful (bracket matching, undo functionality)

Practice

- Valid Parentheses; implement a stack using a plain list

Day 6 — Recursion

Learn

- Base case + recursive case, how the call stack works

Practice

- Factorial, Fibonacci, sum of a list — then trace execution in Python Tutor

Day 7 — Review + Mixed Practice

Learn

- Revisit the week's weakest topic

Practice

- Redo 2-3 earlier problems from memory, without notes

Day 8 — Sorting Algorithms

Learn

- Bubble sort (inefficient) vs. merge/quick sort (efficient), conceptually

Practice

- Implement bubble sort by hand, then compare against Python's built-in sorted()

Day 9 — Searching Algorithms

Learn

- Linear vs. binary search, why binary search needs sorted data

Practice

- Implement binary search from scratch, then 3 LeetCode problems (tag: Binary Search)

Day 10 — Trees (Intro)

Learn

- What a binary tree is; in-order, pre-order, post-order traversal

Practice

- Build a simple tree class and write a traversal function

Day 11 — Binary Search Trees

Learn

- BST insert/search logic and why it beats a plain list

Practice

- 3-4 Easy BST problems

Day 12 — Graphs (Intro)

Learn

- Nodes/edges, adjacency lists, BFS vs. DFS

Practice

- Implement BFS and DFS on a small graph

Day 13 — Dynamic Programming (Intro)

Learn

- What memoization means and why it avoids repeated work

Practice

- Fibonacci with memoization; climbing stairs problem

Day 14 — Final Review + Mock Practice

Learn

- Consolidate the two weeks

Practice

- Solve 5 random Easy/Medium problems across all topics under a time limit; note recurring weak spots

Phase 2 — Intermediate Patterns (Weeks 3-8)

Goal: recognize and solve the algorithm patterns that show up constantly in real problems and interviews.

Week 3-4: Trees & Graphs, Deeper

- Tree traversal variations: level-order (BFS), diameter, lowest common ancestor
- Graph algorithms: topological sort, cycle detection, connected components
- Shortest path basics: Dijkstra's algorithm (conceptual + implementation)
- Practice: 20-25 Medium problems tagged Tree / Graph on LeetCode

Week 5-6: Dynamic Programming

- 1D DP: climbing stairs, house robber, coin change
- 2D DP: unique paths, longest common subsequence, edit distance
- Knapsack pattern and its variations
- Practice: 20 Medium DP problems, re-solving 5 of them from scratch a few days later

Week 7: Greedy Algorithms

- What makes a problem 'greedy' — local optimum leading to global optimum
- Interval scheduling, activity selection, jump game
- Practice: 10-12 Medium greedy problems

Week 8: Backtracking

- Systematic trial-and-error with pruning: permutations, combinations, subsets
- N-Queens, Sudoku solver (conceptual walkthrough), word search
- Practice: 10-12 Medium backtracking problems

Phase 3 — Advanced Structures & Interview Readiness (Months 3-4)

Advanced Data Structures

- Heaps / priority queues — Python's heapq module, top-K problems
- Tries — prefix trees for autocomplete/search-style problems
- Union-Find (Disjoint Set) — connectivity and cycle detection at scale
- Segment trees & Fenwick trees (introductory level)

Advanced Algorithm Patterns

- Advanced DP: bitmask DP, DP on trees, DP with state compression
- Advanced graphs: Bellman-Ford, Floyd-Warshall, minimum spanning tree (Kruskal/Prim)
- Bit manipulation tricks

Interview Readiness

- Practice explaining your approach out loud before coding (mirrors real interviews)
- Time yourself: 25-35 minutes per Medium problem, 40-50 for Hard
- Mock interviews with a peer, or record yourself talking through a solution
- Study system design basics — most senior interviews combine both algorithms and design

Phase 4 — Applied Practice (Months 5-8)

This is where algorithms stop being an abstract exercise and become a real engineering skill — the difference between knowing an algorithm and knowing when to reach for it.

Apply algorithms to real code

- Go back through your Django/full-stack projects (like Green Path) and find places to apply what you've learned: optimizing a slow query loop, caching repeated lookups with a hashmap, replacing nested loops with better data structures
- Profile your own code (Python's cProfile / timeit) to find real bottlenecks, not hypothetical ones

Open source & real codebases

- Read algorithm-heavy Python libraries (e.g. parts of CPython's own standard library, or a small open-source project) to see production-quality implementations
- Make small contributions — bug fixes and small features force you to read and reason about someone else's algorithmic choices

Competitive programming

- Start weekly contests on LeetCode or Codeforces — even placing poorly at first builds real speed and pattern recognition
- Review every contest afterward: which problems did you miss, and why?

Phase 5 — Expert Track (Months 9-12+)

Expertise beyond this point is mostly about volume, depth, and teaching — there's no course that finishes this phase for you.

- Specialize: pick an area to go deep — competitive programming (Codeforces rating climbing), systems/performance engineering, or algorithmic research
- Teach what you know: writing explanations (blog posts, your tech content videos) forces a level of clarity that passive learning never does
- Read a proper algorithms textbook cover to cover for rigor — 'Introduction to Algorithms' (CLRS) or 'Grokking Algorithms' for a gentler version
- Keep a running log of every hard problem you solve and how — patterns repeat, and your own notes become your best resource
- Revisit old problems every few months without looking at your old solution — real mastery shows up as it getting easier, not just familiar

Free Resources Used Throughout

Resource	Best for
freeCodeCamp	Structured Python + DSA course with certificate
CS50 (Harvard, via edX/YouTube)	Deep conceptual understanding, completely free
GeeksforGeeks	Quick reference for any algorithm/data structure, with Python code
LeetCode (free tier)	The industry-standard problem bank, filterable by topic/difficulty
HackerRank	Beginner-friendly structured tracks
Codewars	Bite-sized daily practice problems ('katas')
VisuAlgo	Visual, animated explanations of sorting/searching/trees/graphs
Python Tutor	Step-by-step visualization of your own code executing
Codeforces	Competitive programming contests for Phase 4-5

Honest Expectations

There's no fixed day count that turns anyone into an "expert" — this is a realistic, evidence-based path, not a shortcut. With consistent 4-hour days:

- Strong foundation in core algorithms & data structures: about 1 month
- Comfortable solving Easy/Medium problems independently: about 2 months
- Interview-ready for most companies: about 3-4 months
- Genuinely expert-level, with real depth: 1-2+ years of continued, real practice

Your Django background is a real head start — you already think in terms of how software fits together. This curriculum builds the second half of that skill set: knowing exactly how and why an algorithm should be chosen.